



unc



FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

TRABAJO PRÁCTICO N° 3a

Interfaz de Firmware Extensible Unificada (UEFI)

Integrantes:

- García, Lautaro Misael
- Gomez, Dolores
- Renaudo Gaggioli, Valentino

Mayo 2026

Introducción

TP 1: Exploración del entorno UEFI y la Shell

En este trabajo se exploró el entorno UEFI utilizando QEMU con firmware OVMF. Se analizaron dispositivos, memoria, variables NVRAM y la arquitectura basada en handles y protocolos.

Se utilizó QEMU junto con OVMF para simular un entorno UEFI:

```
qemu-system-x86_64 -m 512 -bios /usr/share/ovmf/OVMF.fd -net none
```

Exploración del sistema

Dispositivos Comando: map

Se observó la presencia de dispositivos de tipo BLK, sin sistemas de archivos montados (FS), lo que evidencia que UEFI no asume automáticamente la existencia de particiones accesibles.

Handles y Protocolos Comando: dh -b

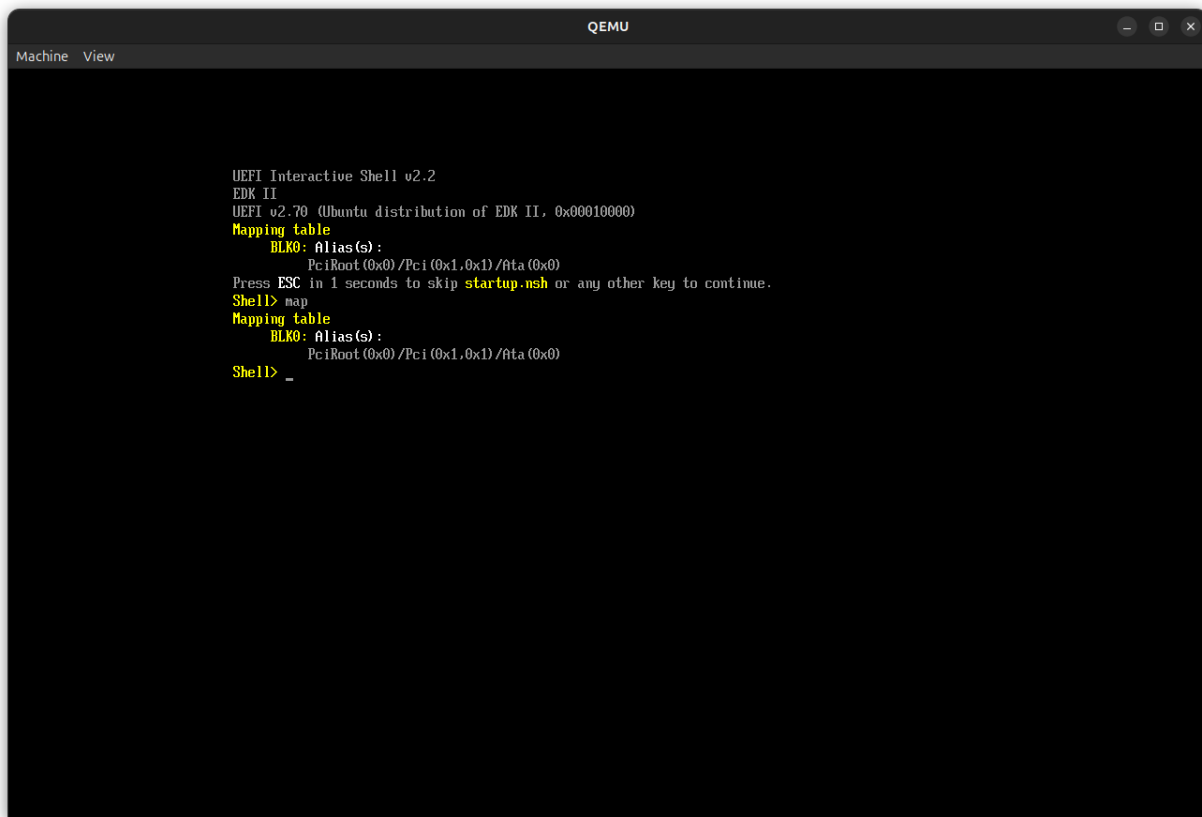
Se verificó que UEFI utiliza un modelo basado en handles y protocolos para representar dispositivos y servicios, en lugar de direcciones de hardware fijas.

Variables NVRAM Comando: dmpstore -b

Se analizaron variables como `BootOrder` y `Boot####`, observando que el orden de arranque se define mediante un arreglo de identificadores almacenado en formato binario.

Ejemplo observado: 00 00 01 00

Interpretación:



```
Machine View
QEMU

UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (Ubuntu distribution of EDK II, 0x00010000)
Mapping table
BLK0: Alias(s):
PciRoot(0x0)/Pci(0x1.0x1)/Ata(0x0)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> map
Mapping table
BLK0: Alias(s):
PciRoot(0x0)/Pci(0x1.0x1)/Ata(0x0)
Shell> _
```

Figure 1: Salida del comando `map`, mostrando dispositivos BLK y FS detectados por UEFI

```
Machine View

Handle dump
01: LoadedImage (DxeCore)
02: Decompress
03: FirmwareVolume2 DevicePath(..F8EB-4F34-AAEA-3EE4AF6516A1) FirmwareVolumeBlock
04: EE4E5898-3914-4259-9D6E-DC7BD79403CF
05: ImageDevicePath(..AD6B-4F3A-B60B-F59899003443) LoadedImage (DevicePathDxe)
06: DevicePathFromText DevicePathToText DevicePathUtilities
07: ImageDevicePath(..87AB-47F9-A3FE-D50B76D89541) LoadedImage (PcdDxe)
08: GetPcdInfo GetPcdInfoProtocol Pcd Pcd
09: BB00A5CA-08CE-462F-A537-43C7A825CA4 ImageDevicePath(..7BCE-40AF-B5BB-47E3611EB5C2) LoadedImage (TdxDxe)
0A: ImageDevicePath(..A7EB-4730-8CBE-CC466A9ECC3C) LoadedImage (ReportStatusCodeRouterRuntimeDxe)
0B: SmartCardReader RscHandler
0C: ImageDevicePath(..43B7-47B4-95B1-F4226CB40CEE) LoadedImage (RuntimeDxe)
0D: RuntimeArch
0E: ImageDevicePath(..7FD6-4665-8646-88E33EF71DFC) LoadedImage (SecurityStubDxe)
0F: SecurityArch Security2Arch
10: DeferredImageLoad
11: DebugSupport 96F46153-97A7-4793-ACC1-F019BF78EA97 EBCInterpreter ImageDevicePath(..73D0-11D4-B06B-00AA00BD6DE7) LoadedImage (EbcDxe)
12: AAEACCFD-F27B-4C17-B610-75CA1F2DFB52
13: ImageDevicePath(..C1BC-49FB-875F-54A5D542443F) LoadedImage (CpuIo2Dxe)
14: CpuIo2
15: ImageDevicePath(..9517-440E-9FDE-3BE44CEE2136) LoadedImage (CpuDxe)
16: CpuArch
17: MpService
18: IncompatiblePciDeviceSupport ImageDevicePath(..A776-4EE1-B643-1FEFF2B615BB) LoadedImage (IncompatiblePciDeviceSupportDxe)
19: PciHotPlugInit ImageDevicePath(..A9BE-426D-A6CC-B22FE51D9224) LoadedImage (PciHotPlugInitDxe)
1A: ImageDevicePath(..FF36-4E10-93CF-A82159E777C5) LoadedImage (ResetSystemRuntimeDxe)
Press ENTER to continue or 'Q' break: _
```

Figure 2: Salida de `dh -b`, donde se observan handles y protocolos asociados a cada dispositivo

```

Machine View

Variable NU*RT*BS 'EFIGlobalVariable:BootOrder' DataSize = 0x04
00000000: 00 00 01 00 .....*
Variable NU*RT*BS 'EFIGlobalVariable:Boot0001' DataSize = 0x58
00000000: 01 00 00 00 2C 00 45 00 49 00 20 00 49 00 .....E.F.I..I.*
00000010: 6E 00 74 00 65 00 72 00 6E 00 61 00 6C 00 20 00 ..n.t.e.r.n.a.l..*
00000020: 53 00 68 00 65 00 6C 00 6C 00 00 00 04 07 14 00 ..S.h.e.l.l.....*
00000030: C9 DD B8 7C EB F8 34 4F-AA EA 3E E4 AF 65 16 A1 ...l..40..>..e..*
00000040: 04 06 14 00 83 A5 04 7C-3E 9E 1C 4F AD 65 E0 52 .....l>..D.e.R*
00000050: 68 D0 B4 D1 7F FF 04 00-.....*h.....*
Variable NU*RT*BS 'EFIGlobalVariable:ErrOut' DataSize = 0x49
00000000: 02 01 0C 00 D0 41 03 0A-00 00 00 01 01 06 00 .....A.....*
00000010: 00 01 02 01 0C 00 D0 41-01 05 00 00 00 03 0E .....A.....*
00000020: 13 00 00 00 00 00 00 C2-01 00 00 00 00 08 01 .....*
00000030: 01 03 0A 14 00 D6 A0 15-AD EC 8B CF 4A A0 73 D0 .....J.s.*
00000040: 1D E7 7E 2D 88 7F FF 04-00.....*
Variable NU*RT*BS 'EFIGlobalVariable:ConIn' DataSize = 0x7A
00000000: 02 01 0C 00 D0 41 03 0A-00 00 00 01 01 06 00 .....A.....*
00000010: 00 01 02 01 0C 00 D0 41-03 03 00 00 00 07 F0 .....A.....*
00000020: 04 00 02 01 0C 00 D0 41-03 0A 00 00 00 01 01 .....A.....*
00000030: 06 00 00 01 02 01 0C 00-D0 41 01 05 00 00 00 00 .....A.....*
00000040: 03 0E 13 00 00 00 00 00-00 C2 01 00 00 00 00 00 .....*
00000050: 08 01 01 03 0A 14 00 D6-A0 15 AD EC 8B CF 4A A0 .....J.*
00000060: 73 D0 1D E7 7E 2D 88 7F-01 04 00 03 0F 0B 00 FF .....s.....*
00000070: FF FF FF 03 01 01 7F FF-04 00 .....*
Variable NU*RT*BS 'EFIGlobalVariable:ConOut' DataSize = 0x67
00000000: 02 01 0C 00 D0 41 03 0A-00 00 00 01 01 06 00 .....A.....*
00000010: 00 01 02 01 0C 00 D0 41-01 05 00 00 00 03 0E .....A.....*
00000020: 13 00 00 00 00 00 00 C2-01 00 00 00 00 08 01 .....*
00000030: 01 03 0A 14 00 D6 A0 15-AD EC 8B CF 4A A0 73 D0 .....J.s.*
00000040: 1D E7 7E 2D 88 7F 01 04-00 02 01 0C 00 D0 41 03 .....A.*
Press ENTER to continue or 'Q' break:

```

Figure 3: Salida de `dmpstore -b`, mostrando variables como `BootOrder` y `Boot####`

- Boot0000
- Boot0001

Memoria Comando: `memmap -b`

Type	Start	End	# Pages	Attributes
BS_Code	0000000000000000-00000000000000FF	0000000000000001	000000000000000F	
Available	0000000000010000-000000000009FFFF	00000000000009F	000000000000000F	
Available	0000000000010000-000000000009FFFF	0000000000000700	000000000000000F	
ACPI_NVS	0000000000080000-0000000000080FFF	0000000000000008	000000000000000F	
Available	0000000000080000-0000000000080FFF	0000000000000003	000000000000000F	
ACPI_NVS	0000000000080000-0000000000080FFF	0000000000000001	000000000000000F	
Available	0000000000080000-0000000000080FFF	0000000000000004	000000000000000F	
ACPI_NVS	0000000000010000-000000000008FFFF	00000000000000F0	000000000000000F	
BS_Data	0000000000090000-00000000000177FFF	00000000000000E0	000000000000000F	
Available	000000000001780000-000000000001BE74FFF	0000000000001A3F5	000000000000000F	
BS_Data	000000000001BE75000-000000000001BE74FFF	0000000000000020	000000000000000F	
Available	000000000001BE75000-000000000001E3CBFFF	00000000000002B37	000000000000000F	
LoaderCode	000000000001E3CC000-000000000001E4A4FFF	00000000000000D9	000000000000000F	
Available	000000000001E4A5000-000000000001E4E2FFF	000000000000003E	000000000000000F	
BS_Data	000000000001E4E3000-000000000001E65CFFF	0000000000000018A	000000000000000F	
BS_Code	000000000001E65D000-000000000001E74DFFF	000000000000000E1	000000000000000F	
BS_Data	000000000001E74E000-000000000001E7B9FFF	0000000000000006C	000000000000000F	
BS_Code	000000000001E7BA000-000000000001E7C2FFF	00000000000000095	000000000000000F	
BS_Data	000000000001E7C3000-000000000001E7C7FFF	00000000000000005	000000000000000F	
BS_Code	000000000001E7C8000-000000000001E806FFF	0000000000000003F	000000000000000F	
BS_Data	000000000001E807000-000000000001E809FFF	00000000000000003	000000000000000F	
BS_Code	000000000001E80A000-000000000001E80CFFF	00000000000000003	000000000000000F	
BS_Data	000000000001E80D000-000000000001E813FFF	00000000000000007	000000000000000F	
BS_Code	000000000001E814000-000000000001E825FFF	00000000000000012	000000000000000F	
BS_Data	000000000001E826000-000000000001E826FFF	00000000000000001	000000000000000F	
BS_Code	000000000001E827000-000000000001E829FFF	00000000000000003	000000000000000F	
BS_Data	000000000001E82A000-000000000001E82EFFF	00000000000000005	000000000000000F	
BS_Code	000000000001E82F000-000000000001E83CFFF	0000000000000000E	000000000000000F	
BS_Data	000000000001E83D000-000000000001E84BFFF	0000000000000000F	000000000000000F	

Press ENTER to continue or 'Q' break: _

Figure 4: Mapa de memoria UEFI, incluyendo regiones RuntimeServices

Se identificaron regiones como **RuntimeServices**, las cuales permanecen activas incluso después de que el sistema operativo toma control.

Drivers Comando: `drivers -b`

Se observó que el firmware UEFI está compuesto por múltiples módulos (drivers), evidenciando una arquitectura modular.

Pregunta de Razonamiento 1:

Pregunta de Razonamiento 1: Al ejecutar el comando `map` y `dh`, vemos protocolos e identificadores en lugar de puertos de hardware fijos. ¿Cuál es la ventaja de seguridad y compatibilidad de este modelo frente al antiguo BIOS?

El modelo de UEFI basado en handles y protocolos actúa como un “middleware” que abstrae por completo el hardware físico. En el BIOS Legacy, el código dependía de interrupciones rígidas y accesos a puertos de I/O específicos (hardcodeados), lo que generaba conflictos y ataba el software a una arquitectura específica. Con UEFI, se interactúa a través de interfaces estandarizadas y orientadas a objetos. Esto no solo garantiza la portabilidad entre distintas plataformas (x86,

```
Machine View

ACPI_Recl 000000001F76D000-000000001F77EFFF 0000000000000012 000000000000000F
ACPI_NUS 000000001F77F000-000000001F77EFFF 0000000000000000 000000000000000F
BS_Data 000000001F7FF000-000000001FDFFFFF 00000000000000601 000000000000000F
Available 000000001FE00000-000000001FE76FFF 0000000000000077 000000000000000F
BS_Data 000000001FE77000-000000001FE96FFF 0000000000000020 000000000000000F
BS_Code 000000001FE97000-000000001FEC9FFF 0000000000000033 000000000000000F
BS_Data 000000001FECB000-000000001FEDAFFF 0000000000000011 000000000000000F
BS_Code 000000001FEDB000-000000001FEF3FFF 0000000000000019 000000000000000F
Press ENTER to continue or 'Q' break:
RT_Data 000000001FEF4000-000000001FF77FFF 0000000000000084 000000000000000F
ACPI_NUS 000000001FF78000-000000001FFFFFFF 0000000000000088 000000000000000F
Reserved 000000FD00000000-000000FFFFFFF 0000000000300000 0000000000000000

Reserved : 3,145,856 Pages (12,885,426,176 Bytes)
LoaderCode: 217 Pages (888,832 Bytes)
LoaderData: 0 Pages (0 Bytes)
BS_Code : 850 Pages (3,481,600 Bytes)
BS_Data : 8,470 Pages (34,693,120 Bytes)
RT_Code : 256 Pages (1,048,576 Bytes)
RT_Data : 581 Pages (2,379,776 Bytes)
ACPI_Recl : 18 Pages (73,728 Bytes)
ACPI_NUS : 513 Pages (2,101,248 Bytes)
MMIO : 0 Pages (0 Bytes)
MMIO_Port : 0 Pages (0 Bytes)
PalCode : 0 Pages (0 Bytes)
Unaccepted: 0 Pages (0 Bytes)
Available : 119,943 Pages (491,286,528 Bytes)
Persistent: 0 Pages (0 Bytes)

Total Memory: 511 MB (535,953,408 Bytes)
Shell> _
```

Figure 5: Salida de `memmap -b`, mostrando la distribución de memoria UEFI con sus distintas regiones y el espacio ocupado por cada tipo

```

Machine View

T D
V C I
R P F A
U VERSION E G G #D #C DRIVER NAME IMAGE NAME
=====
59 0000000A B - - 1 5 PCI Bus Driver PciBusDxe
5B 00000010 ? - - - Virtio PCI Driver VirtioPciDeviceDxe
5C 00000010 ? - - - Virtio 1.0 PCI Driver Virtio10
5D 00000010 ? - - - Virtio Block Driver VirtioBlkDxe
5E 00000010 ? - - - Virtio SCSI Host Driver VirtioScsiDxe
5F 00000010 ? - - - Virtio Random Number Generator Dri VirtioRngDxe
60 00000010 ? - - - Virtio Serial Driver VirtioSerialDxe
61 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
62 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
63 0000000A B - - 2 2 Console Splitter Driver ConSplitterDxe
64 0000000A ? - - - Console Splitter Driver ConSplitterDxe
65 0000000A ? - - - Console Splitter Driver ConSplitterDxe
66 0000000A B - - 2 2 Console Splitter Driver ConSplitterDxe
67 0000000A B - - 1 1 Console Splitter Driver ConSplitterDxe
6B 0000000A D - - 1 - Graphics Console Driver GraphicsConsoleDxe
6C 0000000A B - - 1 1 Serial Terminal Driver TerminalDxe
6D 0000000A D - - 1 - Generic Disk I/O Driver DiskIoDxe
6E 0000000B ? - - - Partition Driver (MBR/GPT/El Torito) PartitionDxe
71 0000000A B - - 1 1 SCSI Bus Driver ScsiBus
72 0000000A D - - 1 - Scsi Disk Driver ScsiDisk
73 0000000A D - - 1 - Sata Controller Init Driver SataController
74 00000010 D - - 1 - AtapiPassThru Driver AtapiPassThruDxe
75 00000010 D - - 1 - ATA Bus Driver AtaBusDxe
76 00000010 ? - - - NVM Express Driver NvmExpressDxe
77 00000010 B - - 1 3 UMF Sio Bus Driver SioBusDxe
Press ENTER to continue or 'Q' break:

```

Figure 6: Listado de drivers cargados en el firmware UEFI

ARM), sino que a nivel de seguridad es vital: obliga a cualquier driver o aplicación a pasar por APIs controladas y validadas por el firmware, evitando accesos directos y arbitrarios a recursos críticos del hardware.

Pregunta de Razonamiento 2:

Observando las variables `Boot####` y `BootOrder`, ¿cómo determina el Boot Manager la secuencia de arranque?

El Boot Manager determina la secuencia de arranque utilizando la variable `BootOrder`, que contiene una lista ordenada de identificadores. Cada uno corresponde a una variable `Boot####`, que incluye un Device Path hacia el ejecutable `.efi`. El firmware recorre esta lista secuencialmente; si una opción falla o el dispositivo no está presente, pasa automáticamente al siguiente identificador hasta lograr transferir el control, eliminando la vieja e insegura práctica del BIOS de ejecutar a ciegas el primer sector (MBR) del disco.

Pregunta de Razonamiento 3:

En el mapa de memoria (`memmap`), existen regiones marcadas como `RuntimeServicesCode`. ¿Por qué estas áreas son un objetivo principal para los desarrolladores de malware (`Bootkits`)?

Las regiones `RuntimeServices` permanecen accesibles después del arranque del sistema operativo, lo que las convierte en un objetivo crítico para ataques como `bootkits`. Estas permiten mantener código con altos privilegios y dificultan su detección desde el sistema operativo.

TP 2: Desarrollo, Compilación y Análisis de Seguridad

Desarrollo de la Aplicación UEFI

Se desarrolló una aplicación nativa en C (`aplicacion.c`) que utiliza la tabla de sistema para imprimir en consola y contiene un breakpoint estático (`INT3`).

Pregunta de Razonamiento 4:

¿Por qué utilizamos `SystemTable->ConOut->OutputString` en lugar de la función `printf` de C?

Se debe a que estamos desarrollando para un entorno de pre-arranque o freestanding (compilado con la bandera `-ffreestanding`). La función `printf` pertenece a la biblioteca estándar de C (`libc`), la cual asume y requiere que haya un Sistema Operativo ejecutándose por debajo para proveer llamadas al sistema que envíen los caracteres a la pantalla.

Al ejecutar una aplicación UEFI, el Sistema Operativo aún no existe en memoria. Por lo tanto, no contamos con `libc`. Para imprimir en pantalla, debemos interactuar directamente con la API que nos provee el firmware a través de la Tabla de Sistema (`SystemTable`), utilizando el protocolo de salida de texto simple (`ConOut`) para enviar cadenas de caracteres Unicode (`L"..."`) nativas del entorno UEFI.

Proceso de Compilación y Automatización

Para la generación del binario `.efi`, se utilizó un script de automatización `build.sh` que realiza los siguientes pasos:

1. Compilación a código objeto con `gcc` (flags: `-fpic`, `-ffreestanding`, etc.).
2. Enlace con `ld` utilizando el script de linker de EFI.
3. Conversión de formato con `objcopy` a `efi-app-x86_64`.

Análisis con Ghidra

Se importó el binario `aplicacion.efi` en Ghidra para analizar la lógica de seguridad y el comportamiento de los opcodes.


```
vale@ThinkBook-14-G8-IRL: ~/repos/SdC-TP-2026/TP3a/src
vale@ThinkBook-14-G8-IRL:~/repos/SdC-TP-2026/TP3a/src$ cd ~/repos/SdC-TP-2026/TP3a/src/
vale@ThinkBook-14-G8-IRL:~/repos/SdC-TP-2026/TP3a/src$ ./build.sh
[1/3] Compilando aplicacion.c a código objeto...
[2/3] Linkeando y generando objeto compartido (.so)...
[3/3] Convirtiendo a ejecutable PE/COFF (.efi)...
¡Éxito! Archivo aplicacion.efi generado correctamente.
vale@ThinkBook-14-G8-IRL:~/repos/SdC-TP-2026/TP3a/src$ ls
aplicacion.c  aplicacion.efi  aplicacion.o  aplicacion.so  build.sh
vale@ThinkBook-14-G8-IRL:~/repos/SdC-TP-2026/TP3a/src$
```

Figure 7: Generación del archivo .efi mediante script

Captura de pantalla:

```
Decompile: efi_main - (aplicacion.efi)
1
2 undefined8 efi_main(longlong param_1)
3
4 {
5     longlong unaff_RSI;
6
7     InitializeLib(param_1);
8     (**(code **)(*(longlong *) (unaff_RSI + 0x40) + 8))(u_Iniciando_analisis_de_seguridad_0000b000);
9     if (mi_breakpoint == -0x34) {
10         (**(code **)(*(longlong *) (unaff_RSI + 0x40) + 8))(u_Breakpoint_estatico_alcanzado_0000b050);
11     }
12     return 0;
13 }
14
```

Figure 8: Función efi_main en Ghidra

Pregunta de Razonamiento 5:

En el pseudocódigo de Ghidra, la condición `0xCC` suele aparecer como `-52`. ¿A qué se debe este fenómeno y por qué importa en ciberseguridad?

El fenómeno -52: Se debe a un problema de interpretación de signos. El valor hexadecimal `0xCC` en binario es `1100 1100`. El motor de descompilación de Ghidra asume que las variables de 1 byte son del tipo con signo (`signed char`), lee el bit más significativo como el bit de signo negativo. Al aplicar la regla del complemento a dos para 8 bits, el valor lógico de `1100 1100` resulta en el número decimal `-52`.

Importancia en Ciberseguridad: El valor `0xCC` corresponde a la instrucción `INT3`, que se utiliza para establecer software breakpoints reemplazando el código en la dirección objetivo y generando la excepción `EXCEPTION_BREAKPOINT`. El malware escanea activamente la memoria en busca del byte `0xCC` para detectar si el código está siendo analizado por un depurador, permitiéndole evadir el análisis (técnicas anti-debugging).

Nota: Para lograr que el descompilador mostrara el chequeo, fue necesario declarar la variable como `volatile` en el scope global, forzando a las herramientas a leer la memoria asumiendo que un agente externo o hardware

podría haberla modificado.

```
#include <efi.h>
#include <efilib.h>

// DECLARACIÓN GLOBAL Y VOLÁTIL
volatile unsigned char mi_breakpoint = 0xCC;

EFI_STATUS efi_main(...) { ... }
```

TP 3: Ejecución en Hardware Físico (Bare Metal)

Esta sección documenta el procedimiento para preparar el medio de arranque, validarlo en un entorno virtual y ejecutarlo en hardware físico. Además, se detalla una refactorización crítica del código debida a incompatibilidades a nivel de arquitectura.

Refactorización del Código y Análisis del Fallo de `OutputString`

En la propuesta original de la cátedra, se sugería utilizar el protocolo nativo de UEFI para imprimir texto en pantalla mediante la siguiente instrucción:

```
SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Mensaje\r\n");
```

Durante las pruebas, se comprobó que esta instrucción **provocaba un cuelgue total (congelamiento) del sistema**, tanto en el emulador como en el hardware físico. Por este motivo, se optó por refactorizar el código utilizando la función de alto nivel `Print(L"Mensaje\r\n");` (el código fuente actualizado puede consultarse en el repositorio).

Investigación del fallo (ABI Mismatch): El problema no radica en un error de sintaxis, sino en una incompatibilidad a nivel de la Interfaz Binaria de Aplicación y la Convención de Llamadas:

1. **El estándar UEFI** fue desarrollado con gran influencia de Microsoft, por lo que utiliza la **Microsoft x64 Calling Convention**. En este estándar, los primeros argumentos de una función se pasan a través de los registros `RCX` y `RDY` del procesador.
2. **Nuestro entorno de compilación** utiliza `gcc` en Linux, el cual asume por defecto la convención **System V AMD64 ABI**. Aquí, los primeros argumentos se pasan en los registros `RDI` y `RSI`.

Al compilar nuestro código en Linux e intentar ejecutar la llamada directa a `OutputString`, nuestro binario guarda los punteros del texto en `RDI`. Sin embargo, el firmware de UEFI va a buscar esos punteros a `RCX` (donde en ese momento hay basura o ceros). Esto provoca que el procesador intente leer memoria inválida, generando una excepción no manejada (*Page Fault*) que congela la máquina.

Solución: La función `Print` provista por la biblioteca (`efilib`) actúa como un *wrapper*. Internamente se encarga de reordenar los registros del procesador desde el formato System V al formato Microsoft de manera dinámica antes de llamar al firmware, logrando una ejecución estable y segura.

Procedimiento de Preparación del Pendrive

Para que el firmware UEFI reconozca un dispositivo extraíble, este debe contar obligatoriamente con una tabla de particiones válida y un sistema de archivos **FAT32**.

Paso a paso en Linux:

1. Mediante el comando `lsblk` se identificó el dispositivo USB (ej. `/dev/sda`) y se generó una nueva tabla de particiones con `fdisk`. Se creó una partición primaria asignándole el tipo EFI (FAT-12/16/32).
2. Se formateó la nueva partición (`/dev/sda1`) en FAT32:

```
sudo mkfs.fat -F32 /dev/sda1
```

3. Se montó la partición y se recreó la estructura de directorios exigida por la especificación UEFI:

```
sudo mount /dev/sda1 /mnt
sudo mkdir -p /mnt/EFI/BOOT
```

4. Se descargó la UEFI Shell oficial de TianoCore y se la renombró como el cargador por defecto (BOOTX64.EFI), ubicándola en la carpeta correspondiente.
5. Se copió nuestro binario compilado (aplicacion.efi) directamente en la raíz del pendrive (/mnt/) para facilitar su ejecución desde la Shell. Finalmente, se desmontó la unidad de forma segura.

Validación en Entorno Virtual (QEMU)

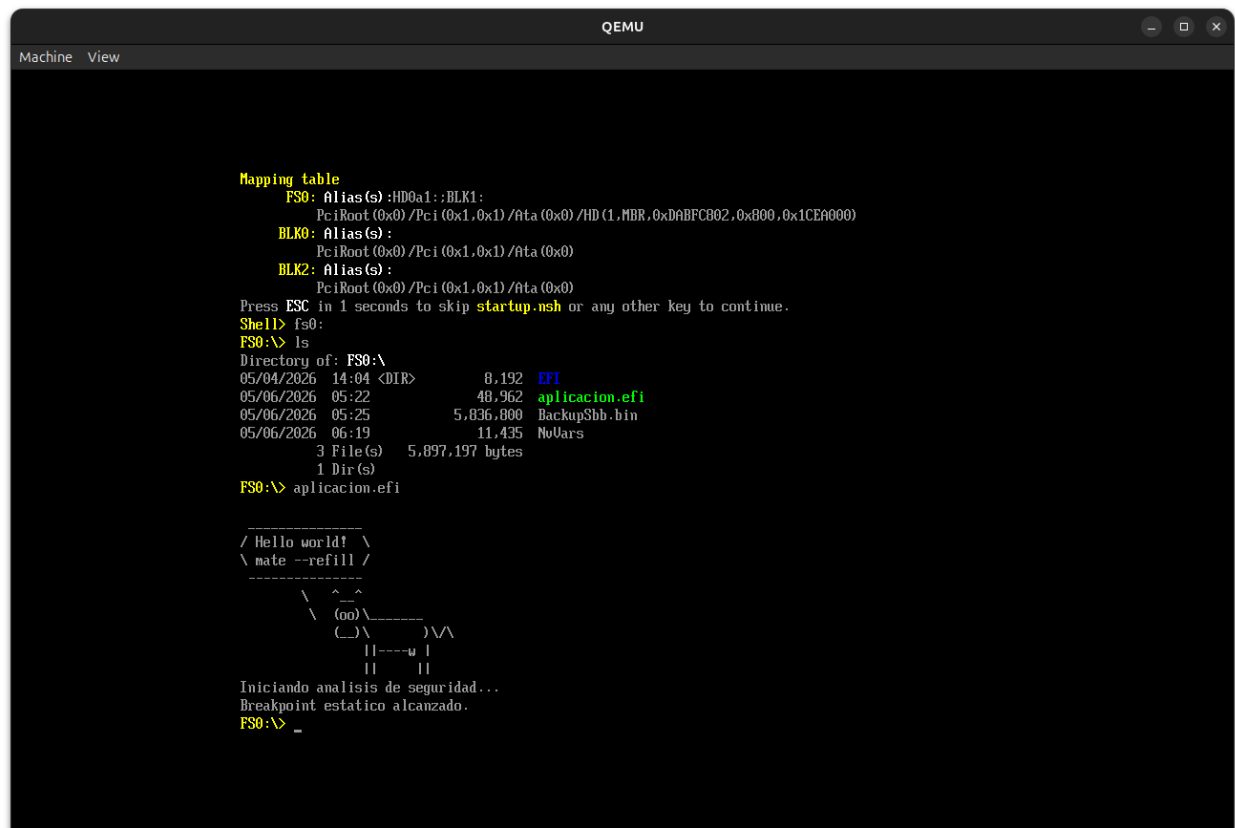
Antes de someter el código al hardware físico, se validó la correcta configuración de la partición y el funcionamiento del binario utilizando QEMU con el firmware OVMF.

En lugar de crear una imagen de disco virtual, se mapeó el pendrive físico directamente a la máquina virtual utilizando el siguiente comando (requiere privilegios de superusuario para leer el dispositivo de bloques):

```
sudo qemu-system-x86_64 -bios /usr/share/ovmf/OVMF.fd -drive file=/dev/sda,format=raw
```

Esta prueba permitió comprobar que la UEFI Shell iniciaba automáticamente y que el comando Print mostraba los caracteres de forma correcta sin congelar el sistema.

Captura de validación en QEMU:



```
Machine View

Mapping table
FS0: Alias(s) : HD0a1::BLK1:
      PciRoot (0x0) / Pci (0x1,0x1) / Ata (0x0) / HD (1,MBR,0xDA8FC802,0x800,0x1CEA000)
BLK0: Alias(s) :
      PciRoot (0x0) / Pci (0x1,0x1) / Ata (0x0)
BLK2: Alias(s) :
      PciRoot (0x0) / Pci (0x1,0x1) / Ata (0x0)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> fs0:
FS0:\> ls
Directory of: FS0:\
05/04/2026  14:04 <DIR>          8,192  EFI
05/06/2026  05:22          48,962  aplicacion.efi
05/06/2026  05:25       5,836,800  BackupSbb.bin
05/06/2026  06:19         11,435  NoVars
          3 File(s)  5,897,197 bytes
          1 Dir(s)
FS0:\> aplicacion.efi

/ Hello world! \
\ mate --refill /
-----
  \  ^  ^
   \ (oo)\_____ )\
    (__)\       )\/\
       ||----w |
       ||     ||

Iniciando analisis de seguridad...
Breakpoint estatico alcanzado.
FS0:\> _
```

Figure 9: Validación en QEMU

Ejecución en Hardware Físico

Para la prueba final, se insertó el pendrive en una notebook física (Lenovo).

1. Se ingresó a la configuración del BIOS/UEFI para **desactivar la opción Secure Boot**. Este paso es necesario, ya que nuestros binarios (BOOTX64.EFI y aplicacion.efi) fueron compilados localmente y no poseen las firmas criptográficas validadas por el fabricante.
2. Desde el menú de arranque, se seleccionó la unidad USB.
3. Al iniciar la UEFI Shell, se identificó la unidad correspondiente al medio extraíble en la tabla de mapeo (en este caso, FS1:).
4. Se navegó hacia la unidad y se ejecutó la aplicación tipeando aplicacion.efi.

El programa se ejecutó correctamente en el nivel de firmware pre-OS, validando de forma empírica la condición estática de interrupción y enviando el mensaje por pantalla.

Captura de ejecución en hardware físico:

```
UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (Phoenix Technologies Ltd., 0x12345678)
Mapping table
FS2: Alias(s) :HD2b0a1:;BLK8:
PciRoot(0x0)/Pci(0x8,0x2)/Pci(0x0,0x0)/Sata(0x1,0x0,0x0)/HD(1,GPT,72504F7F-92D7-4CE1-84FD-097DF7FA737C,0x000,0x32000)
FS0: Alias(s) :HD0b:;BLK1:
PciRoot(0x0)/Pci(0x1,0x1)/Pci(0x0,0x0)/NVMe(0x1,05-36-E6-39-68-B7-26-00)/HD(1,GPT,B1F0A469-496C-49DA-80E7-A4BDFB61505C,0x000,0x32000)
FS1: Alias(s) :HD1d0b:;BLK6:
PciRoot(0x0)/Pci(0x8,0x1)/Pci(0x0,0x3)/USB(0x3,0x0)/HD(1,MBR,0xDABFCB02,0x000,0x1CE000)
BLK7: Alias(s) :
PciRoot(0x0)/Pci(0x8,0x2)/Pci(0x0,0x0)/Sata(0x1,0x0,0x0)
BLK9: Alias(s) :
PciRoot(0x0)/Pci(0x8,0x2)/Pci(0x0,0x0)/Sata(0x1,0x0,0x0)/HD(2,GPT,303981D6-342E-406D-BD20-55A9C3C0CB65,0x32000,0x8000)
BLK10: Alias(s) :
PciRoot(0x0)/Pci(0x8,0x2)/Pci(0x0,0x0)/Sata(0x1,0x0,0x0)/HD(3,GPT,A97F897F-E745-4021-A5CF-A7E52AFEAF3F,0x30000,0x745C0250)
BLK11: Alias(s) :
PciRoot(0x0)/Pci(0x8,0x2)/Pci(0x0,0x0)/Sata(0x1,0x0,0x0)/HD(4,GPT,26CDB22-8BE9-47A3-94AF-CF600EB1ABE7,0x74603000,0x103000)
BLK0: Alias(s) :
PciRoot(0x0)/Pci(0x1,0x1)/Pci(0x0,0x0)/NVMe(0x1,05-36-E6-39-68-B7-26-00)
BLK2: Alias(s) :
PciRoot(0x0)/Pci(0x1,0x1)/Pci(0x0,0x0)/NVMe(0x1,05-36-E6-39-68-B7-26-00)/HD(2,GPT,55DE0C99-76E0-400C-B002-9D49DF20C5E9,0x32000,0x8000)
BLK3: Alias(s) :
PciRoot(0x0)/Pci(0x1,0x1)/Pci(0x0,0x0)/NVMe(0x1,05-36-E6-39-68-B7-26-00)/HD(3,GPT,54E0A55D-B78B-4064-9F7E-9F1EBD9080BC,0x30000,0x1D006A50)
BLK4: Alias(s) :
PciRoot(0x0)/Pci(0x1,0x1)/Pci(0x0,0x0)/NVMe(0x1,05-36-E6-39-68-B7-26-00)/HD(4,GPT,7660B343-11CC-4DE1-A0E2-19BAD93E14B0,0x1D041000,0x1B3000)
BLK5: Alias(s) :
PciRoot(0x0)/Pci(0x8,0x1)/Pci(0x0,0x3)/USB(0x3,0x0)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> fs1:
FS1:\> ls
Directory of: FS1:\
05/04/2026 14:04 <DIR>          8,192 EFI
05/06/2026 05:22              48,962 aplicacion.efi
05/06/2026 05:25          5,836,800 BackupSbb.bin
                2 File(s)    5,805,762 bytes
                1 Dir(s)
FS1:\> aplicacion.efi

/ Hello world! \
\ mate --refill /

-----
  \  ^__^
   (oo)\_______
      (__)\       )\/\
         ||----w |
         ||     ||

Iniciando analisis de seguridad...
Breakpoint estatico alcanzado.
FS1:\>
```

Figure 10: Ejecución en hardware físico

Conclusiones

El desarrollo y análisis llevado a cabo en este trabajo práctico demuestra que la Interfaz de Firmware Extensible Unificada (UEFI) no es una simple evolución del BIOS heredado, sino un ecosistema operativo robusto y complejo que se ejecuta antes de la carga del Sistema Operativo.

Desde la perspectiva del desarrollo, la programación nativa en UEFI exige comprender las limitaciones de un entorno *freestanding*, donde la ausencia de bibliotecas estándar como `libc` obliga a interactuar directamente con las estructuras y protocolos del firmware (como la `SystemTable` y la convención de llamadas *Microsoft ABI*). A su vez, la abstracción del hardware mediante identificadores y protocolos dota al firmware de una portabilidad sin precedentes, desterrando la rigidez de los antiguos puertos de E/S fijos.

Sin embargo, esta misma flexibilidad y complejidad amplía drásticamente la superficie de ataque. La permanencia de las regiones de memoria `RuntimeServices` ilustra cómo el diseño del firmware puede ser explotado por bootkits para alcanzar la cúspide de la persistencia y la evasión, operando de manera invisible para el kernel del OS.

Finalmente, el análisis de ingeniería inversa con Ghidra pone en evidencia que el análisis de binarios de bajo nivel está fuertemente influenciado por las optimizaciones del compilador (que requerimos evadir mediante variables globales `volatile`) y la interpretación matemática del descompilador. Comprender fenómenos como el *signedness* al evaluar opcodes como `INT3` (`0xCC` / `-52`) es una habilidad indispensable en ciberseguridad para diseñar reglas de detección certeras, entender el comportamiento de herramientas *anti-debugging* y mitigar vulnerabilidades antes de que el Sistema Operativo siquiera tome el control del hardware.